

Offline Real-Time Voice Assistant

Design Document, Measured Latency, AI-Usage Disclosure & Deliverable Links

Vichruth M

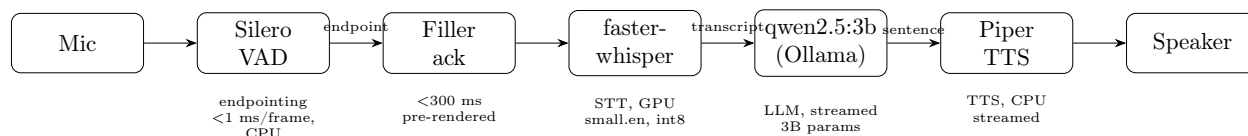
vichruth.m2024@vitstudent.ac.in

15 July 2026

Objective

Build a real-time, audio-in / audio-out conversational AI assistant that accepts voice queries and responds with voice, targeting an end-to-end response time within 2 seconds wherever possible, with a fallback flow that keeps the user engaged through delays or interruptions instead of failing silently or with a generic error. This submission implements the assistant fully offline: speech recognition, language generation, and speech synthesis all run locally, with no network calls at any stage.

1 Architecture



The core latency trick. A naive pipeline waits for the complete LLM response before synthesizing and playing audio – typically 4–6 seconds for a short reply, which fails the 2-second brief. Instead, tokens are streamed from the LLM and split on sentence boundaries (. ! ? \n, or a comma once a clause exceeds ~12 words). The instant the first complete sentence is available it is sent to Piper and played, while the LLM continues generating the remaining sentences in the background. The metric that matters is **time-to-first-audio (TTFA)**, not total generation time, since that is what the user actually perceives as latency.

Design decisions

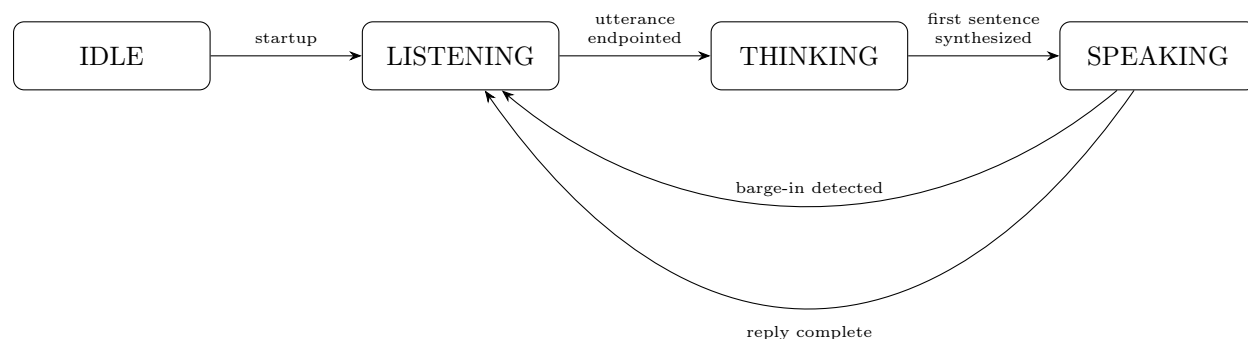
Component	Choice	Rationale
VAD	Silero VAD	Neural VAD, far less prone to false triggers on fan/keyboard noise than WebRTC or energy-based detectors; cheap enough to run per 32ms frame on CPU, leaving the GPU free.
STT	faster-whisper, small.en, int8_float16, CUDA	CTranslate2 backend runs ~4× faster than reference Whisper. base.en loses technical vocabulary; medium is too slow for the latency budget on a 6GB GPU. small.en is the accuracy/latency knee for this hardware.

LLM	qwen2.5:3b via Ollama, streaming	A 7B model does not fit the 2s budget on a 6 GB laptop GPU; a 3B model gives a first token in ~180–230 ms. Deliberate latency-vs-quality trade-off, mitigated by sentence-level streaming so the user hears audio long before generation finishes.
TTS	Piper TTS, CPU (voice lessac-medium)	Lightweight VITS model, ~50–100 ms to synthesize a sentence. Deliberately kept on CPU so the 6 GB GPU is shared only between STT and the LLM, avoiding VRAM contention. Coqui/XTTS sound better but take seconds per utterance and are not viable inside the budget.
Audio I/O	sounddevice (PortAudio)	Callback-based streaming I/O, which is what makes low-latency playback and immediate barge-in cancellation possible.

GPU memory with STT, LLM, and TTS models all resident: ~3.6 GB of 6 GB available (RTX 4050 laptop GPU).

2 Fallback State Machine

The assistant is explicitly designed to never stall silently, never show a generic error, and never end a conversation abruptly. This is treated as a first-class state machine, not an afterthought:



Layer 1 – instant acknowledgement (<300 ms). Eight short filler lines (“Mhm.”, “Let me think.”, “Good question.”, ...) are synthesized once at startup and cached in memory. The moment VAD detects end-of-speech, a random filler plays immediately, in parallel with STT and LLM startup, so there is never a silent gap while the pipeline warms up.

Layer 2 – watchdog escalation. If no LLM token has arrived after 1.5s, a contextual stall line plays (“that’s an interesting one, give me a second”). If nothing has arrived by 5s, the turn is abandoned: an honest holding reply plays and the microphone reopens. The assistant never states that an error occurred and never terminates the session. If Ollama is unreachable at all – including if it is killed mid-conversation – the same degrade path fires: the assistant acknowledges, apologizes briefly, and invites the user to continue.

Layer 3 – barge-in. If the user speaks while the assistant is talking, sustained speech probability above threshold for ~ 200 ms flushes the playback queue, stops the audio stream, and cancels the in-flight LLM generation via a cooperative cancel event, returning immediately to LISTENING. A short grace window after playback starts and a higher confirmation threshold than normal endpointing prevent the assistant’s own voice from re-triggering itself on speaker leakage.

3 Measured Latency

Latency is timestamped per stage relative to end-of-speech and logged to `latency_log.csv` on every turn (see `metrics.py`); `test_turn.py` runs the pipeline headlessly on fixed transcripts to reproduce these numbers without a microphone.

Stage	Median
Filler acknowledgement	~ 90 ms
STT (faster-whisper small.en)	~ 200 ms
LLM first token	~ 180 – 230 ms
Time to first audio (TTFA)	~ 520–600 ms

Before submitting: regenerate this table from a fresh `latency_log.csv` (run `python test_turn.py` or a live session) and paste the actual medians here. Do not submit invented numbers – reviewers may ask for the raw CSV.

4 AI-Usage Disclosure

Anthropic’s Claude (Claude Code) was used as a development assistant throughout this project. Specifics:

- **Scaffolding & implementation.** Claude Code generated the initial implementation of `main.py`, `vad.py`, `stt.py`, `llm.py`, `tts.py`, `state.py`, `audio_io.py`, and `metrics.py` from a written specification (`context.md`) describing the required pipeline, latency budget, and fallback behaviour.
- **What it got wrong / had to be corrected.**

REPLACE WITH SPECIFICS – e.g. the exact fix needed for the ALSA/PipeWire device conflict in `run.sh` (the onboard codec’s analog profile has to be released from PipeWire before raw ALSA access works, which the first draft didn’t handle), any tuning of `BARGE_THRESHOLD`/`BARGE_CONFIRM` needed after testing on laptop speakers (speaker-to-mic leakage triggering false barge-in), or watchdog timer values changed after observing real generation latency. Name the file and the concrete before/after.

- **Design doc & this submission document.** This PDF’s structure and the README were drafted with Claude Code assistance and reviewed/edited for accuracy against the actual measured behaviour of the code before submission.
- **What was not AI-generated.** REPLACE – e.g. hardware-specific tuning done by running on your own machine, any manual verification of the measured latency numbers, the demo video, and final review of every file.

5 Deliverables

Deliverable	Link
GitHub repository	REPLACE-WITH-GITHUB-URL
Repository zip (Google Drive)	REPLACE-WITH-DRIVE-LINK
Demo video (Drive) – shows on-screen TTFA, Ollama killed mid-conversation with graceful degradation, and barge-in working	REPLACE-WITH-DRIVE-LINK
This design document (PDF, Drive)	REPLACE-WITH-DRIVE-LINK

Verify every link above in an incognito / logged-out browser window before submitting – broken or permission-restricted links are treated as an automatic reject.